RETAILMART V3 · SQL

Date/Time Mastery for Time-Series Analytics

— Siraj Sir

📱 WhatsApp Announcement

Date/Time Mastery for Time-Series Analytics

Time-series data is the backbone of every dashboard. Today you learn the PostgreSQL tools that make time-series queries clean and correct: generate_series for date dimensions, gap-filling for sparse data, and timezone handling.

Part 1: generate_series -- Creating Date Dimensions
Part 2: Gap-Filling -- Zeros for Missing Days
Part 3: Time Bucketing and Timezone Awareness

After today, you can build 'daily revenue for the last 90 days' with zero gaps, bucket events by hour to find peak times, and handle UTC-to-IST conversions correctly.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: generate_series for Dates	2 Hours	Generating date ranges, Building a date dimension table, Fiscal calendar fields
Part 2: Gap-Filling		LEFT JOIN against a date series, Zeros for missing days, Complete daily/weekly/monthly reports
Part 3: Time Bucketing & Timezones		DATE_TRUNC for hourly/daily/weekly, TIMESTAMPTZ and AT TIME ZONE, UTC to IST conversion

YOUR SQL JOURNEY

- ✓ Window Funcs Part 1
- ✓ Aggregation & Frames
- ✓ LAG & LEAD
- ✓ Query Plans
- ✓ Indexing
- ✓ Pivoting & FILTER
- ✓ Median & DISTINCT ON
- 23 **Date/Time Mastery** ← HERE

RECAP

What We Covered Yesterday

- PERCENTILE_CONT for median and P90/P95 percentiles
- DISTINCT ON for latest-per-group pattern
- Executive KPI strip combining multiple aggregation patterns

STORY

The Missing Days Problem

You run a daily revenue query and get 88 rows for 90 days. Two days had zero orders and are missing from the result. Your line chart shows a straight line through those days instead of a dip to zero. The fix: generate ALL 90 days first, then LEFT JOIN your data. generate_series creates the complete date backbone.

DEFINITION

generate_series(start, stop, step)

- Produces a set of values from start to stop, incrementing by step
- For dates: generate_series('2025-01-01'::DATE, '2025-12-31'::DATE, '1 day'::INTERVAL)
- Step can be: '1 day', '1 week', '1 month', '1 hour', etc.
- Returns one row per generated value

Basic generate_series

```sql
-- Generate all days in January 2025:
SELECT d::DATE AS day
FROM generate_series(
  '2025-01-01'::DATE,
  '2025-01-31'::DATE,
  '1 day'::INTERVAL
) AS d;

-- Returns 31 rows: 2025-01-01 through 2025-01-31
```

Date Dimension Table (1/2)

```
SELECT
  d::DATE                                AS day,
  EXTRACT(DOW FROM d)                    AS day_of_week,
  TO_CHAR(d, 'Day')                      AS day_name,
  EXTRACT(WEEK FROM d)                   AS week_num,
  EXTRACT(MONTH FROM d)                  AS month_num,
  TO_CHAR(d, 'Month')                    AS month_name,
  EXTRACT(QUARTER FROM d)                AS quarter,
  EXTRACT(YEAR FROM d)                   AS year,
  CASE
    WHEN EXTRACT(MONTH FROM d) <= 3
      THEN 'Q' || EXTRACT(QUARTER FROM d)
          || '-FY' || EXTRACT(YEAR FROM d)
    ELSE 'Q' || (EXTRACT(QUARTER FROM d) - 3)
        || '-FY' || (EXTRACT(YEAR FROM d) + 1)
  END                                    AS fiscal_quarter
FROM generate_series(
  '2020-01-01'::DATE,
  '2030-12-31'::DATE,
  '1 day'::INTERVAL
```

Date Dimension Table (2/2)

```
) AS d;
```

PRO TIP

PRO TIP

generate_series is a set-returning function. Use it in the FROM clause like a table. You can also use it with timestamps and intervals for hourly or minute-level series.

STORY

The Complete Daily Revenue Report

The CFO's dashboard shows daily revenue for the last 90 days. If day 45 had no orders, it should show 0, not skip that day. Gap-filling is the pattern: generate all dates, LEFT JOIN your data, and COALESCE nulls to zero.

Gap-Fill Template (1/2)

```
-- Step 1: Generate the complete date backbone
-- Step 2: LEFT JOIN your data onto it
-- Step 3: COALESCE NULLs to 0

WITH date_spine AS (
  SELECT d::DATE AS day
  FROM generate_series(
    CURRENT_DATE - INTERVAL '90 days',
    CURRENT_DATE,
    '1 day'::INTERVAL
  ) AS d
),
daily_revenue AS (
  SELECT
    order_date::DATE  AS day,
    SUM(net_total) AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
    AND order_date >= CURRENT_DATE - INTERVAL '90 days'
  GROUP BY order_date::DATE
```

Gap-Fill Template (2/2)

```
)
SELECT
  ds.day,
  COALESCE(dr.revenue, 0) AS revenue
FROM date_spine ds
LEFT JOIN daily_revenue dr ON ds.day = dr.day
ORDER BY ds.day;
```

This always returns exactly 91 rows (90 days + today), even if some days had zero orders.

Weekly Series (1/2)

```
WITH week_spine AS (
  SELECT d::DATE AS week_start
  FROM generate_series(
    DATE_TRUNC('week', CURRENT_DATE - INTERVAL '12 weeks'),
    DATE_TRUNC('week', CURRENT_DATE),
    '1 week'::INTERVAL
  ) AS d
),
weekly_data AS (
  SELECT
    DATE_TRUNC('week', order_date)::DATE AS week_start,
    COUNT(*)         AS orders,
    SUM(net_total) AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY DATE_TRUNC('week', order_date)
)
SELECT
  ws.week_start,
  COALESCE(wd.orders, 0)  AS orders,
```

Weekly Series (2/2)

```
  COALESCE(wd.revenue, 0) AS revenue
FROM week_spine ws
LEFT JOIN weekly_data wd ON ws.week_start = wd.week_start
ORDER BY ws.week_start;
```

Practice 1: Date Dimension for 2025

EASY

SCENARIO: The analytics team needs a reusable date dimension table.

TASK: Generate a date dimension for all of 2025 with columns: day, day_of_week (0=Sun..6=Sat), week_num, month_name, quarter, is_weekend (boolean).

HINT: EXTRACT(DOW FROM d) for day of week. DOW 0 and 6 are weekend days.

Answer

✅ ANSWER

```
SELECT
  d::DATE                        AS day,
  EXTRACT(DOW FROM d)::INT       AS day_of_week,
  EXTRACT(WEEK FROM d)::INT      AS week_num,
  TO_CHAR(d, 'Mon')              AS month_name,
  EXTRACT(QUARTER FROM d)::INT   AS quarter,
  EXTRACT(DOW FROM d) IN (0, 6)  AS is_weekend
FROM generate_series(
  '2025-01-01'::DATE,
  '2025-12-31'::DATE,
  '1 day'::INTERVAL
) AS d;
```

Practice 2: Gap-Filled Daily Revenue

MEDIUM

SCENARIO: Build a complete daily revenue report for the last 30 days with no missing days.

TASK: Use generate_series and LEFT JOIN to produce 31 rows of daily revenue. Days with no orders should show 0. Include a running total column.

HINT: Gap-fill first with COALESCE, then add SUM() OVER (ORDER BY day) for the running total.

Answer (1/2)

✅ ANSWER

```
WITH spine AS (
  SELECT d::DATE AS day
  FROM generate_series(
    CURRENT_DATE - 30,
    CURRENT_DATE,
    '1 day'::INTERVAL) AS d
),
daily AS (
  SELECT order_date::DATE AS day,
         SUM(net_total) AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
    AND order_date >= CURRENT_DATE - 30
  GROUP BY order_date::DATE
),
filled AS (
  SELECT s.day,
    COALESCE(d.revenue, 0) AS revenue
  FROM spine s
  LEFT JOIN daily d ON s.day = d.day
)
SELECT day, revenue,
```

Answer (2/2)

✅ ANSWER

```
  SUM(revenue) OVER (ORDER BY day) AS cumulative
FROM filled
ORDER BY day;
```

Hourly Bucketing

Find peak shopping hours by bucketing web page views into 1-hour windows. NOTE: order_date is a DATE (no time-of-day), so hour-of-day analysis must use a real TIMESTAMP column like page_views.view_timestamp.

Hourly Buckets

```
SELECT
  EXTRACT(HOUR FROM view_timestamp) AS hour_of_day,
  COUNT(*)                          AS view_count
FROM web_events.page_views
GROUP BY EXTRACT(HOUR FROM view_timestamp)
ORDER BY hour_of_day;
```

Fine-Grained Bucketing

```
-- Bucket web events into 15-minute windows:
SELECT
  DATE_TRUNC('hour', view_timestamp)
    + INTERVAL '15 min'
    * FLOOR(EXTRACT(MINUTE FROM view_timestamp) / 15)
    AS bucket,
  COUNT(*) AS events
FROM web_events.page_views
GROUP BY bucket
ORDER BY bucket;
```

DEFINITION

TIMESTAMP vs TIMESTAMPTZ

- TIMESTAMP: stores date+time with NO timezone info
- TIMESTAMPTZ: stores as UTC internally, converts on display
- AT TIME ZONE: converts between timezones

UTC to IST Conversion

```
-- Convert UTC timestamps to IST (India Standard Time):
SELECT
  view_timestamp                        AS utc_time,
  view_timestamp AT TIME ZONE 'UTC'
    AT TIME ZONE 'Asia/Kolkata'         AS ist_time,
  EXTRACT(HOUR FROM
    view_timestamp AT TIME ZONE 'UTC'
    AT TIME ZONE 'Asia/Kolkata')        AS ist_hour
FROM web_events.page_views
LIMIT 5;

-- IST = UTC + 5:30
-- So 10:00 UTC = 15:30 IST
```

PRO TIP

PRO TIP

Always store timestamps as TIMESTAMPTZ (with timezone). Convert to local time only at display time using AT TIME ZONE. This prevents bugs when your servers, databases, and users are in different timezones.

Practice 3: Peak Shopping Hour Analysis

HARD

SCENARIO: The operations team wants to know peak shopping hours to optimize staffing.

TASK: Bucket orders by hour of day (in IST). Show the top 5 busiest hours by order count. Include average order value per hour.

HINT: Convert order_date AT TIME ZONE 'Asia/Kolkata', then EXTRACT(HOUR FROM ...).

Answer

✅ ANSWER

```
SELECT
  EXTRACT(HOUR FROM
    order_date AT TIME ZONE 'Asia/Kolkata')
    AS ist_hour,
  COUNT(*)                          AS orders,
  ROUND(AVG(net_total)::NUMERIC, 2)
                                   AS avg_value
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY ist_hour
ORDER BY orders DESC
LIMIT 5;
```

Practice 4: Revenue by ISO Week with Correctness

HARD

SCENARIO: Finance uses ISO weeks (Mon-Sun). Build a weekly revenue report that handles year boundaries correctly.

TASK: Show ISO year, ISO week number, and revenue. Use DATE_TRUNC('week', ...) for correct ISO week boundaries. Gap-fill any missing weeks.

HINT: EXTRACT(ISOYEAR FROM d) and EXTRACT(WEEK FROM d) give ISO-correct values.

Answer (1/2)

✅ ANSWER

```
WITH week_spine AS (
  SELECT DATE_TRUNC('week', d)::DATE AS week_start
  FROM generate_series(
    DATE_TRUNC('week', '2025-01-01'::DATE),
    DATE_TRUNC('week', CURRENT_DATE),
    '1 week'::INTERVAL) AS d
),
weekly AS (
  SELECT
    DATE_TRUNC('week', order_date)::DATE AS week_start,
    SUM(net_total) AS revenue
  FROM sales.orders
  WHERE order_status = 'Delivered'
  GROUP BY DATE_TRUNC('week', order_date)
)
SELECT
  ws.week_start,
  EXTRACT(ISOYEAR FROM ws.week_start)::INT AS iso_year,
  EXTRACT(WEEK FROM ws.week_start)::INT AS iso_week,
  COALESCE(w.revenue, 0) AS revenue
FROM week_spine ws
LEFT JOIN weekly w ON ws.week_start = w.week_start
```

Answer (2/2)

✅ ANSWER

```
ORDER BY ws.week_start;
```

Practice 5: First-30-Day Cohort Timeline

CRAZY

SCENARIO: Build a timeline showing each customer cohort's activity in their first 30 days after signup.

TASK: For customers who signed up in January 2025, create a day-by-day timeline (day 0 through day 30) showing the number of orders placed on each day relative to signup. Gap-fill with zeros.

HINT: generate_series(0, 30) for day numbers. Cross join with cohort customers. LEFT JOIN orders where order_date = signup_date + day_offset.

Answer (1/2)

✅ ANSWER

```
WITH cohort AS (
  SELECT customer_id, registration_date AS signup
  FROM customers.customers
  WHERE registration_date >= '2025-01-01'
    AND registration_date < '2025-02-01'
),
day_offsets AS (
  SELECT generate_series(0, 30) AS day_num
),
orders_by_offset AS (
  SELECT
    o.order_date::DATE - c.signup AS day_num,
    COUNT(*) AS orders
  FROM cohort c
  JOIN sales.orders o
    ON c.customer_id = o.cust_id
   AND o.order_date::DATE
     BETWEEN c.signup AND c.signup + 30
  GROUP BY o.order_date::DATE - c.signup
)
SELECT
  d.day_num,
```

Answer (2/2)

✅ ANSWER

```
  COALESCE(ob.orders, 0) AS orders
FROM day_offsets d
LEFT JOIN orders_by_offset ob ON d.day_num = ob.day_num
ORDER BY d.day_num;
```

INTERVIEW

Q: How do you handle missing dates in time-series data?

A: Use generate_series to create a complete date backbone, then LEFT JOIN your data onto it. COALESCE null values to zero. This ensures every date appears in the result even if no events occurred that day.

INTERVIEW

Q: What is the difference between TIMESTAMP and TIMESTAMPTZ?

A: TIMESTAMP stores a date-time with no timezone awareness. TIMESTAMPTZ stores internally as UTC and converts on display based on the session timezone. Always use TIMESTAMPTZ for event data. Convert to local time at display time with AT TIME ZONE.

HW 1: Monthly Date Spine

EASY

TASK: Generate a monthly series for all of 2025 (12 rows). Show month_start, month_name, and quarter.

Answer

✅ ANSWER

```
SELECT
  d::DATE                       AS month_start,
  TO_CHAR(d, 'Month')           AS month_name,
  EXTRACT(QUARTER FROM d)::INT  AS quarter
FROM generate_series(
  '2025-01-01'::DATE,
  '2025-12-01'::DATE,
  '1 month'::INTERVAL) AS d;
```

HW 2: Gap-Filled Weekly Orders

MEDIUM

TASK: Build a gap-filled weekly order count for the last 12 weeks. Include week_start, order_count (0 for missing weeks), and cumulative_orders.

Answer

✅ ANSWER

```
WITH spine AS (
  SELECT DATE_TRUNC('week', d)::DATE AS wk
  FROM generate_series(
    DATE_TRUNC('week', CURRENT_DATE - 84),
    DATE_TRUNC('week', CURRENT_DATE),
    '1 week'::INTERVAL) AS d
),
weekly AS (
  SELECT DATE_TRUNC('week', order_date)::DATE AS wk,
         COUNT(*) AS cnt
  FROM sales.orders GROUP BY 1
)
SELECT s.wk,
  COALESCE(w.cnt, 0) AS orders,
  SUM(COALESCE(w.cnt, 0)) OVER (ORDER BY s.wk)
    AS cumulative
FROM spine s LEFT JOIN weekly w ON s.wk = w.wk
ORDER BY s.wk;
```

HW 3: Hourly Page View Heatmap

MEDIUM

TASK: Count page views by hour of day (IST) and day of week. This creates a heatmap-ready dataset.

Answer

✅ ANSWER

```
SELECT
  EXTRACT(DOW FROM
    view_timestamp AT TIME ZONE 'Asia/Kolkata')
    ::INT AS dow,
  TO_CHAR(view_timestamp
    AT TIME ZONE 'Asia/Kolkata', 'Dy') AS day_name,
  EXTRACT(HOUR FROM
    view_timestamp AT TIME ZONE 'Asia/Kolkata')
    ::INT AS hour,
  COUNT(*) AS views
FROM web_events.page_views
GROUP BY 1, 2, 3
ORDER BY dow, hour;
```

HW 4: Monthly Revenue Gap-Fill by Region

HARD

TASK: For each region, produce monthly revenue for 2025 H1. Gap-fill with 0 for months where a region had no delivered orders.

Answer (1/2)

✅ ANSWER

```
WITH months AS (
  SELECT d::DATE AS m FROM generate_series(
    '2025-01-01'::DATE, '2025-06-01'::DATE,
    '1 month'::INTERVAL) AS d
),
regions AS (
  SELECT region_name AS region FROM core.dim_region
),
skeleton AS (
  SELECT r.region, m.m AS month
  FROM regions r CROSS JOIN months m
),
actual AS (
  SELECT dr.region_name AS region,
    DATE_TRUNC('month', o.order_date)::DATE AS month,
    SUM(o.net_total) AS rev
  FROM sales.orders o
  JOIN stores.stores s ON o.store_id = s.store_id
  JOIN core.dim_region dr ON s.region_id = dr.region_id
  WHERE o.order_status = 'Delivered'
    AND o.order_date >= '2025-01-01'
    AND o.order_date < '2025-07-01'
```

Answer (2/2)

✅ ANSWER

```
  GROUP BY 1, 2
)
SELECT sk.region, sk.month,
  COALESCE(a.rev, 0) AS revenue
FROM skeleton sk
LEFT JOIN actual a
  ON sk.region = a.region AND sk.month = a.month
ORDER BY sk.region, sk.month;
```

HW 5: 15-Minute Page-View Buckets

HARD

TASK: Find the peak 15-minute window of the day by page-view count. Show the top 10 busiest windows. NOTE: orders are DATE-only, so sub-day buckets use page_views.view_timestamp.

Answer

✅ ANSWER

```
SELECT
  DATE_TRUNC('hour', view_timestamp)
    + INTERVAL '15 min'
    * FLOOR(EXTRACT(MINUTE FROM view_timestamp)/15)
    AS bucket_15m,
  COUNT(*) AS views
FROM web_events.page_views
GROUP BY bucket_15m
ORDER BY views DESC
LIMIT 10;
```

HW 6: Customer Cohort Retention Grid

CRAZY

TASK: Build a monthly cohort retention grid. Rows = signup month. Columns = month 0, 1, 2, … 5. Values = count of customers who placed an order in that month offset.

Answer (1/2)

✅ ANSWER

```
WITH cohort AS (
  SELECT customer_id,
    DATE_TRUNC('month', registration_date)::DATE AS signup_m
  FROM customers.customers
  WHERE registration_date >= '2025-01-01'
    AND registration_date < '2025-07-01'
),
activity AS (
  SELECT c.signup_m, c.customer_id,
    (DATE_TRUNC('month', o.order_date)::DATE
     - c.signup_m) / 30 AS month_offset
  FROM cohort c
  JOIN sales.orders o
    ON c.customer_id = o.cust_id
  WHERE o.order_date >= c.signup_m
    AND o.order_date < c.signup_m + INTERVAL '6 months'
)
SELECT signup_m,
  COUNT(DISTINCT customer_id)
    FILTER (WHERE month_offset = 0) AS m0,
  COUNT(DISTINCT customer_id)
    FILTER (WHERE month_offset = 1) AS m1,
```

Answer (2/2)

✅ ANSWER

```
  COUNT(DISTINCT customer_id)
    FILTER (WHERE month_offset = 2) AS m2,
  COUNT(DISTINCT customer_id)
    FILTER (WHERE month_offset = 3) AS m3,
  COUNT(DISTINCT customer_id)
    FILTER (WHERE month_offset = 4) AS m4,
  COUNT(DISTINCT customer_id)
    FILTER (WHERE month_offset = 5) AS m5
FROM activity
GROUP BY signup_m
ORDER BY signup_m;
```

KEY TAKEAWAYS

generate_series creates date backbones: Use it to generate complete date/week/month sequences for gap-filling.

Gap-fill with LEFT JOIN + COALESCE: LEFT JOIN data onto a date spine. COALESCE(value, 0) fills missing periods with zero.

DATE_TRUNC for time bucketing: Bucket by hour, day, week, month, quarter with a single function.

Always use TIMESTAMPTZ: Store as UTC, display as local time with AT TIME ZONE.

CROSS JOIN for multi-dimensional spines: Regions x Months creates a complete skeleton for gap-filling by group.

What is Next

Tomorrow we explore JSON and semi-structured data: JSONB operators, jsonb_array_elements, and flattening nested JSON -- essential when your data arrives from APIs and event streams.